

Devicer NEXT

A Near-Everything Comparison Engine

Technical Whitepaper - Version 3.0.0

Gateway Corporate LLC

October 2026

Publication scope. This paper describes the model-free NEXT engineering preview in the 3.0.0 codebase. It distinguishes implemented contracts, recorded verification, and research goals. “Near-everything” describes an extensible comparison interface, not universal recognition accuracy or support for arbitrary inputs without a domain adapter. No production ML implementation or weights are included in the release scope.

Previous Whitepaper

FP-Devicer: Open-Source Digital Fingerprinting Middleware (Version 2.0.3)

Legacy architecture and algorithms on which this NEXT paper builds.

gatewaycorporate.org/papers/FP-Devicer.pdf

Table of Contents

1	Abstract	3
2	Motivation And Design Principles	3
2.1	From Device Scores To Declared Relationships	3
2.2	Non-Negotiable Distinctions	3
3	Architecture And Public Surfaces	4
3.1	Two Explicit API Contracts	4
3.2	Extension And Isolation	4
4	Observations, Provenance, And Evidence	5
4.1	Observation Contract	5
4.2	Configuration Binding	5
4.3	Evidence And Raw Fusion	5
5	Baseline Adapters And Signal Profiles	5
5.1	Browser And Document Baselines	5
5.2	Physical Fingerprint Acquisition	6
5.3	Signature Normalization	6
5.4	Faces, Feature Vectors, And Invalid Input	6
6	Calibration And Selective Decisions	7
6.1	Empirical Calibration Baseline	7
6.2	Prediction Sets And Their Limits	7
6.3	Evaluation And Adaptation	7
7	Retrieval, Persistence, And Artifact Lifecycle	8
7.1	Exact Retrieval Is Candidate Generation	8
7.2	Observation Stores	8
7.3	Calibration Artifact Rollout	8
8	Inference And Runtime Boundaries	8
9	Legacy Algorithms And Compatibility	9
9.1	Retained Composite Scorer	9
9.2	History, Drift, Graphs, And LSH	9
9.3	Compatibility And Migration Discipline	10
10	Evaluation And Recorded Evidence	10
10.1	Software Verification	10
10.2	Task-Level Evaluation	10
10.3	Performance Measurement	10
11	Privacy, Security, And Operational Governance	11
12	Reproducibility And Release Boundaries	11
13	Conclusion	11
14	Sources And Implementation References	12

1 Abstract

Devicer NEXT extends FP-Devicer from server-side browser fingerprint scoring into a domain-neutral framework for versioned observations, relationship-scoped comparisons, explicit evidence, and configuration-bound calibration. A common interface allows applications to compare browser signals, identify exact document duplicates, or integrate external feature extractors while retaining the meaning and provenance of each comparison.

The central change is semantic: a content key is not an identity, a similarity is not a probability, and an unavailable observation is not a negative match. NEXT exposes these distinctions through typed results, explicit unsupported and insufficient-data states, and a separate calibration layer. Acquisition profiles make physical input assumptions inspectable: fingerprint ingress defaults to known 500-DPI gray8 acquisition, while signature ingress uses a versioned, aspect-preserving 224-by-224 fit-and-pad profile.

The implementation includes exact retrieval, in-memory and database-backed observation stores, artifact activation and rollback, governance hooks, and application-owned inference runtimes. The existing root API remains separate and retains the legacy composite scorer, device manager, registry, drift, graph, and LSH interfaces. Rust ports and a native compatibility bridge provide an isolated verification path, not a complete replacement for the TypeScript runtime.

Recorded checks establish software behavior and packaged compatibility for a defined corpus. They do not establish population-wide identity accuracy, biometric certification, production-scale throughput, or statistical validity under arbitrary distribution shift. Browser and document workflows are the initial advertised domains; biometric and handwriting surfaces are integration contracts requiring independent validation.

2 Motivation And Design Principles

2.1 From Device Scores To Declared Relationships

The original FP-Devicer paper described a server-side composite scorer and a persistent device-identification workflow. Moving comparison to the server makes history, configurable comparators, and storage available to applications, but it does not make a browser fingerprint a durable or authenticated identity. Commodity configurations collide; browsers change; observations can be missing, privacy-reduced, replayed, or fabricated.

NEXT generalizes the useful engineering structure without generalizing the identity claim. The same pair of inputs can support different questions: whether documents have exactly equal text, whether observations are consistent with the same installation, or whether externally extracted features support a declared enrolled-subject relationship. The adapter names the question before the system computes a score.

2.2 Non-Negotiable Distinctions

- **Observation versus subject:** an observation records supplied features at a time; it is not automatically a person, account, or permanent device.
- **Exact content versus identity:** canonical content equality supports deduplication, not common authorship or ownership.
- **Similarity versus probability:** a bounded comparison score has no universal probabilistic interpretation.
- **Prediction versus decision:** a calibrated estimate and its uncertainty are inputs to an application's risk policy, not a policy by themselves.

- **Unavailable versus false:** missing evidence calls for abstention, additional collection, or review rather than an invented negative observation.
- **Pairwise versus transitive:** two pairwise matches do not justify merging an entire connected component into one identity.

NEXT does not collect signals from browsers or scanners. Applications remain responsible for collection, consent, authentication, enrollment, and any ground truth used to label pairs. The library does not manufacture trusted labels from its own past decisions.

3 Architecture And Public Surfaces

3.1 Two Explicit API Contracts

The package root, `devicer.js`, preserves the legacy API. NEXT is additive through `devicer.js/next`; dedicated subpaths expose SQLite, PostgreSQL, Redis, and the Node process runtime. TypeScript declarations accompany all six public entry points. Generated TypeDoc documentation now includes both API families.

Layer	Responsibility
Adapter	Validate supplied input, create observations, declare relationships, and compute evidence.
Comparison	Check compatible provenance, select available evidence, and produce a bounded raw score.
Calibration	Bind labeled-data artifacts to a configuration and report supported predictions or explicit failure states.
Retrieval	Produce candidates and measure candidate recall independently of pairwise comparison.
Persistence	Store scoped observations and support validated snapshots, restore, and rollback.
Governance	Apply application authorization, retention, and audit callbacks.
Host runtime	Execute JavaScript callbacks or application-owned inference with explicit lifecycle controls.

The logical workflow is collection, validation, observation creation, candidate retrieval, pairwise evidence, calibration, and application decision. Persistence and audit can accompany multiple stages. This is a composition of APIs, not a claim that one NEXT call orchestrates every stage or reproduces all legacy DeviceManager behavior.

3.2 Extension And Isolation

FingerprintAdapter is the main extension boundary. It declares a domain, schema, extractor versions, configuration, supported relationships, an observation factory, and an evidence comparator. InstanceRegistry supports instance-scoped adapter registration and removal. New domains need not modify the comparison core, but their acquisition, comparison, and validation semantics remain the adapter author's responsibility.

The legacy global weight/comparator registry and its post-identification plugin system remain distinct from NEXT adapters. Existing JavaScript plugins are host callbacks; a native library cannot silently replace their execution environment.

4 Observations, Provenance, And Evidence

4.1 Observation Contract

An Observation contains an ID, domain, schema version, observation time, extractor-version map, and feature record. Optional quality, missingness, provenance, and acquisition fields retain contextual information needed to interpret the features. Applications should assign meaningful observation IDs and preserve acquisition times rather than confusing ingestion with capture.

For model-backed integration, provenance records the model manifest, checkpoint digest, preprocessing, runtime, extractor versions, input profile, and declared feature dimension. These are application-supplied descriptors, not evidence that a particular sensor or executable is authentic. A matching manifest is a compatibility check, not remote attestation.

4.2 Configuration Binding

Configuration identifies the domain, schema, extractors, comparators, fusion, normalization, runtime, and, when relevant, model and signal profile. A digest provides a stable identifier for the declared pipeline. Before scoring, NEXT checks observation domain, schema, extractor versions, and required model, profile, and dimension provenance against the adapter.

Incompatible observations return `unsupported_configuration`. Missing metadata is not silently upgraded to the newest version. Migration must use known provenance or re-extract from retained source data. Changing preprocessing, dimensions, comparator behavior, or model artifacts may change the score distribution even when the application considers the task unchanged.

4.3 Evidence And Raw Fusion

An evidence item identifies the relationship, both observation IDs, comparator version, similarity, and availability. Optional explanations describe unavailable signals; dependency groups help describe related evidence. Dependency metadata does not itself remove correlation from the fusion rule.

Let A be the set of available evidence items with finite scores. The current generic fusion rule is an arithmetic mean after bounding each score:

$$s = \frac{\sum_{i \in A} \min(1, \max(0, s_i))}{|A|}.$$

If A is empty, the result has similarity: `null` and status `insufficient_data`. Otherwise raw comparison returns `uncalibrated`. Duplicating correlated evidence can influence a mean; learned dependency-aware fusion is not an implemented guarantee of this baseline.

`matchFingerprint` selects the highest-scoring comparable supplied candidate and applies an explicit minimum similarity. It can return `match`, `non_match`, or `insufficient_evidence`. It does not perform calibration automatically. Passing a calibration-status label is not a substitute for applying a validated artifact.

5 Baseline Adapters And Signal Profiles

5.1 Browser And Document Baselines

The browser adapter evaluates exact agreement among jointly defined fields for the same-installation relationship. With comparable field set K , its score is the fraction whose values agree. No jointly available fields means no comparison, not perfect agreement. This deliberately simple baseline is not the legacy weighted recursive/TLSH scorer.

The document adapter compares exact text for exact-content, yielding one for equal text and zero otherwise. It does not perform OCR, language normalization, semantic embedding, or authorship attribution. An application can provide a different adapter when those operations are needed and independently justified.

5.2 Physical Fingerprint Acquisition

FINGERPRINT_500_DPI identifies `fingerprint.gray8.500dpi.v1`. Its default contract is single-channel gray8 with known 500-DPI acquisition density and valid dimensions and buffer length. DPI describes physical sampling density, not a fixed image size. Relabeling a low-resolution raster as 500 DPI does not restore missing information or certify a scanner.

Alternative raster profiles must explicitly declare DPI and a versioned profile ID. Profile validation is not fingerprint enhancement, minutiae extraction, liveness detection, or compliance with a biometric interchange standard. The repository example uses synthetic sensor-shaped captures and model-free row-band features to exercise integration only.

5.3 Signature Normalization

SIGNATURE_224 identifies `signature.gray8.fit-pad-224.v1`. Applications supply 224-by-224 gray8 rasters produced by aspect-preserving fitting and white padding, together with source dimensions and normalization metadata. For source width w and height h , the intended geometric scale is

$$a = \min\left(\frac{224}{w}, \frac{224}{h}\right).$$

The resized image fits within the target square and is centered with padding; it is not independently stretched along both axes. This is an explicit, versioned normalization profile, not a universal signature industry standard. The TypeScript ingress validates the submitted contract; it does not prove that the caller actually performed the declared preprocessing.

The Rust PNG/JPEG example composites transparency onto white, converts to luminance, rejects blank captures, fits and pads the image, and derives normalized ink values under its own profile. Its pixel-distance baseline is uncalibrated and does not establish writer identity or resistance to forgery.

5.4 Faces, Feature Vectors, And Invalid Input

Face integrations require an explicit raster or feature-only profile. Raster shape, channel count, preprocessing, and alignment assumptions belong to the declared pipeline. Handwriting and enrolled-biometric feature-only factories require an extractor version and dimension. Model-backed vector factories also require declared dimensions; production extraction remains external.

Raster validation checks positive integer dimensions, exact decoded buffer length, profile compatibility, and configured blank-input rejection. Current bounds include an 8192-pixel maximum per dimension and a 32-MiB decoded buffer limit. These checks do not replace safe decoding limits upstream.

Vector comparisons reject missing, sparse, nonfinite, zero-magnitude, or dimensionally invalid features as unavailable evidence. Valid vectors use a numerically scaled cosine calculation, mapped into the unit interval:

$$c = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \sqrt{\sum_i y_i^2}}, \quad s = \frac{1 + c}{2}.$$

Opposite valid vectors can therefore score zero without being invalid. This is a generic feature comparator, not an ISO minutiae matcher. Transcription evidence, when present in the corresponding adapter, uses exact text rather than assumed case-insensitive equivalence.

6 Calibration And Selective Decisions

6.1 Empirical Calibration Baseline

`createCalibrationArtifact` accepts labeled comparison samples, one configuration digest, and a data manifest describing population, task, collection interval, observation IDs, label sources, and calibration split. Labels record pair IDs and an external, analyst, or verified source; those declarations still need operational validation.

The implemented `empirical-bin.v1` method partitions the unit score interval into bins, with ten bins by default. If a bin contains n_b samples and k_b positive labels, its smoothed estimate is

$$\hat{p}_b = \frac{k_b + 1}{n_b + 2}.$$

An empty bin returns 0.5. The estimate is a finite-sample baseline, not a monotonicity constraint, a likelihood ratio, or evidence that the bin has adequate deployment support. It depends on the sampled class balance and population; case-control sampling may require additional treatment before deployment.

6.2 Prediction Sets And Their Limits

The artifact records nonconformity scores equal to $1 - \hat{p}$ for positive labels and $\hat{p}$ for negative labels. A finite-sample rank based on the requested error level selects a quantile. At prediction time, each binary label is included when its nonconformity is no greater than that quantile. Sets may contain one label, both labels, or neither label.

Important statistical boundary: the present implementation estimates bin probabilities and the nonconformity quantile from the same supplied samples. It is a conformal-style engineering baseline, not an independently fitted split-conformal protocol. The code alone does not establish a finite-sample coverage guarantee. Independent fitting/calibration design, valid sampling assumptions, and held-out measurement are required before making such a claim.

`calibrateComparison` checks configuration identity, artifact expiry, comparable data, and the supported evidence mask. Its explicit states include `supported`, `stale`, `unsupported_configuration`, and `insufficient_data`. A supported state means that these artifact checks passed; it does not certify that production data match the declared population or that prediction sets have useful coverage.

6.3 Evaluation And Adaptation

`evaluateCalibration` reports supported and unsupported counts, Brier score, log loss, empirical coverage, and mean prediction-set size. The score metrics are computed on supported predictions; reports must retain the unsupported denominator so exclusion cannot make performance look artificially strong. `evaluateCalibrationByMask` exposes differences across evidence availability.

Use independent held-out data, disjoint subjects or source groups where the task requires them, and separate tuning from reporting. Different manifest IDs alone do not establish independence. Dataset split validation utilities support this discipline but cannot detect all hidden duplicate content or incorrect labels.

`assessCalibrationReuse` rejects reuse across changed configuration digests. Label allocation separates uncertainty-driven review from random audit sampling with recorded selection probabilities.

This helps make sampling bias inspectable; it does not create labels, automatically improve a model, or prove recovery with fewer labels after distribution shift.

The research objective is to test whether adaptation can recover useful, reliable decisions after extractor changes or evidence loss with fewer labels than full recalibration. A defensible experiment compares against full recalibration, no adaptation, and strong fixed baselines on frozen independent evaluation data. No such improvement is claimed by this implementation paper.

7 Retrieval, Persistence, And Artifact Lifecycle

7.1 Exact Retrieval Is Candidate Generation

`createExactContentId` derives a SHA-256 key from canonical content scoped by domain and schema. `ExactRetrievalIndex` supports idempotent observation updates, removal, deterministic ordering, pagination, manifests, validated snapshots, and rebuilds. Exact retrieval uses those content IDs to find candidate observations, separately from application decisions.

The current index is in memory and scans its entries when finding equal keys. A returned-result limit is not a sublinear search guarantee. It is not HNSW, an approximate-nearest-neighbor service, or a persistent identity graph. The exhaustive candidate oracle and exact-recall evaluation help check retrieval behavior without conflating retrieval misses with comparator errors.

7.2 Observation Stores

`NextObservationStore` defines scoped observation operations. In-memory stores serve ephemeral use; persistent stores use a snapshot persistence boundary with SQLite, PostgreSQL, and Redis implementations. Stored payloads retain IDs, timestamps, features, and provenance. Snapshot validation rejects incompatible domain/schema data, and transaction failures are tested for rollback behavior.

The implementation is snapshot-oriented. A durable reopen or rollback test does not prove efficient high-volume writes, isolation among independent writers, distributed transactions, or crash consistency for every deployment. Operators must evaluate their concurrency model, database configuration, backup/restore, and failure modes. Backend names alone do not establish these guarantees.

7.3 Calibration Artifact Rollout

`CalibrationArtifactRegistry` retains registered artifacts and activation history, with defensive reads and rollback by artifact selection. Activation records a reason; rollback selects a prior artifact rather than editing historical observations or labels. The registry is an in-memory foundation, not a durable distributed rollout coordinator.

In a deployment, retain the application version, configuration digest, artifact ID, relevant data manifests, and decision policy together. A reproducible decision needs more than a model name or a single similarity value. Durable registry storage and operational promotion controls remain application responsibilities.

8 Inference And Runtime Boundaries

Generic model-backed adapters accept an application-supplied inference callback or `ModelRuntime`. Manifest fields make the expected model and preprocessing inspectable without bundling an implementation. Validation occurs before inference; cancellation and inference failures remain explicit runtime concerns.

`ProcessModelRuntime`, available from the Node runtime subpath, uses a JSON-lines protocol with lazy startup, serialized requests, bounded timeouts, cancellation, response decoding, and teardown.

The executable, its dependencies, and any model artifacts are owned by the integrating application. Process separation is not a sandbox or protection against a malicious executable.

The shipped branch excludes production model implementations, vendored model repositories, checkpoints, and model-specific setup tooling. Package and source inventory checks enforce that boundary. Synthetic and model-free fixtures remain useful for testing contracts but cannot measure biometric discrimination, liveness, demographic error differences, or robustness to capture conditions.

The Rust workspace ports legacy algorithms and supports an isolated native Node bridge with host-callback and lifecycle tests. Normal TypeScript usage is not automatically Rust-backed. The checked WASM artifact builds and instantiates but exports only memory, with no callable scoring API. The portable-loader test exercises injection mechanics, not native/WASM scoring parity.

9 Legacy Algorithms And Compatibility

9.1 Retained Composite Scorer

The 2.0.3 whitepaper remains the historical reference for the root API. Its recursive weighted structural comparison can be summarized as

$$S_{\text{structural}} = \frac{\sum_i w_i s_i}{\sum_i w_i}.$$

The legacy engine also computes TLSH similarity over canonicalized JSON, with distance normalization constant 300, and blends it with structural similarity. Secondary dimensions include evidence richness, field agreement, structural stability, entropy contribution, attractor risk, and one-sided or two-sided missingness. Nonlinear penalties, an offset, exact-match promotion, and a non-exact ceiling shape the final integer score in the range zero to 100.

The historical term “calibration” in that heuristic scorer is not equivalent to NEXT’s labeled-data probability calibration. Dividing legacy confidence by 100 does not yield a validated NEXT probability. Default thresholds and behavior are legacy compatibility semantics, not new universal recommendations.

9.2 History, Drift, Graphs, And LSH

The legacy device manager combines candidate lookup, historical stability, temporal modulation, scoring, ID assignment, persistence, deduplication, observability, and post-identification plugins. Drift analysis compares current fields with historical variation. The identity graph records heuristic subnet and font-overlap associations; these edges are not proof of a common person. Frequency-table attractor scoring allows operator-supplied commonness estimates.

The legacy MinHash LSH index retrieves candidates over set-valued signals such as fonts and languages. For independent banding with B bands, r rows per band, and Jaccard similarity J , the standard collision model is

$$P_{\text{candidate}} = 1 - (1 - J^r)^B.$$

Its characteristic scale is $B^{-\frac{1}{r}}$, approximately 0.707 for 16 bands of eight rows, not the 0.50 approximation stated in the older paper. This model is not a guarantee for every finite hash implementation or population. Neither the legacy LSH implementation nor its historical throughput figures establish NEXT collection-search scale.

9.3 Compatibility And Migration Discipline

Differential checks use an integrity-pinned published devicer.js@2.0.3 oracle, separate from candidate package archives. Runtime exports, public declarations, consumer behavior, and storage fixtures are checked at package boundaries. This is evidence for the defined corpus, not proof of every earlier 2.x release or all possible JavaScript callback behavior.

Applications can keep root imports while introducing NEXT explicitly. Legacy stored fingerprints must not be relabeled as versioned NEXT observations without known schema and extractor provenance. Migration should preserve IDs and times, validate manifests and row counts, support a dry run, and avoid destructive changes to the source store. Unsupported historical data should remain explicit rather than being assigned invented acquisition metadata.

10 Evaluation And Recorded Evidence

10.1 Software Verification

The release record dated October 4, 2026 reports clean Linux source-snapshot checks on Node 20.20.2, 22.23.3, and 24.21.0, with Rust 1.98.1. It records:

- 346 unit tests passing per Node version, plus build and signal-example checks.
- Three release-policy tests and source/package model-exclusion checks.
- Packaged legacy parity for 42 runtime exports, 76 declarations, and six isolated consumer groups.
- 26 native bridge checks per Node version, strict Clippy, formatting, and nine core plus two example Rust tests.
- Live SQLite, PostgreSQL, and Redis compatibility, including NEXT reopen and rollback checks.
- Inspection of the then-current package archive and successful imports of all six runtime exports.
- A fresh WASM build and load check, restricted to the memory-only artifact described above.

These are attributed historical results from the recorded working-tree checkpoint, not a claim that this paper's final commit or a subsequently packed archive underwent the same checks. Repacking documentation changes the artifact; hashes belong to specific archives. Final-commit CI and dependency-advisory triage remain release gates. The record includes unresolved advisories and is not a security certification.

10.2 Task-Level Evaluation

The NEXT evaluation runner supports pairwise false-match and false-nonmatch rates, abstention reporting, quality slices, and latency percentiles. A deployment report should freeze the task, relation, source groups, population, candidate policy, input profile, extractor, comparator, artifact, and threshold before evaluation. Report sample counts and uncertainty intervals along with rates; state how unavailable cases and abstentions enter each denominator.

For enrolled biometrics or writer verification, split by the relevant subject, writer, session, sensor, and document-source groups to control leakage. Report genuine and impostor composition, enrollment conditions, capture quality, and adversarial scope. Avoid treating synthetic positive and negative labels as evidence of performance on real subjects.

10.3 Performance Measurement

The original whitepaper's scenario scores and cached-path timing artifacts are historical regression evidence, not NEXT benchmarks. Repeated device-manager calls can primarily exercise deduplication, making them inappropriate estimates of cold retrieval, fresh inference, or durable write costs.

Measure each stage and the full workflow with hardware, runtime, concurrency, data volume, candidate counts, warmup, and cache policy declared. Distinguish median and tail latency, throughput

under load, memory growth, and failure behavior. This paper makes no universal accuracy percentage, sub-millisecond latency promise, Rust speedup claim, or automatic improvement-with-use claim.

11 Privacy, Security, And Operational Governance

Fingerprint and biometric observations can be sensitive even when pseudonymous. Content hashes are deterministic identifiers and may permit linkage or guessing; hashing is not anonymization. Collect only necessary signals, document lawful purpose, apply appropriate consent and access controls, and set retention according to actual need. Do not treat similarity as authentication.

`createGovernedNextStore` adds application-defined authorization, retention, and audit hooks. Applications must schedule expiration purges and explicitly record verification or abstention events where appropriate. Audit events omit raw features by design, but identifiers and free-form reasons still need review for sensitive content. Hooks are not a complete identity/access-management system, an encrypted store, or an automatic compliance framework.

Threat analysis should include spoofed acquisition metadata, replayed features, malicious inputs, untrusted model executables, correlated or duplicated evidence, label poisoning, unauthorized artifact promotion, backup retention, and multi-writer persistence races. Resource limits and provenance validation address parts of this surface; they do not prove liveness, input authenticity, or correctness under adversarial distribution shift.

12 Reproducibility And Release Boundaries

The TypeScript API requires an appropriate Node environment and the chosen storage driver's peer dependencies. Rust is needed for bridge development, not for ordinary NEXT JavaScript use. A source checkout can exercise the principal engineering checks with:

```
npm ci --ignore-scripts
npm rebuild better-sqlite3
npm run build
npm test
npm run test:signal-example
npm run test:release
npm run compat:package
npm run next:release:check
npm run docs
```

With a matching Rust toolchain, use `cargo test --workspace --all-targets --locked`, strict Clippy and formatting checks, `npm run bridge:check`, and `npm run bridge:portable`. Live storage checks require configured services and must not silently substitute memory-only fixtures for required backends.

Future release and research work includes broader runtime/OS evidence, any functional WASM API, production-scale persistence and retrieval evaluation, independently evaluated domain adapters, and a statistically defensible adaptation protocol. A useful engineering preview does not by itself demonstrate research novelty, and a negative adaptation result would not invalidate the value of explicit, reproducible comparison contracts.

13 Conclusion

Devicer NEXT's contribution is a disciplined boundary between observations, evidence, calibration, and decisions. It preserves the legacy device workflow while making new comparisons versioned,

relationship-specific, inspectable, and capable of declining unsupported conclusions. Realistic acquisition profiles and external inference interfaces enable integration without shipping production models or implying that an integration fixture is a recognition product.

The path to deployment is task-specific validation, reproducible artifacts, operational controls, and measured uncertainty. Extensibility broadens the kinds of questions an application can ask; evidence determines which answers it can responsibly use.

14 Sources And Implementation References

This paper adapts the architecture and legacy algorithm discussion in the existing `whitepaper.md`. Current implementation takes precedence over historical plans or examples. Repository paths below identify the relevant sources; pin a commit when citing them in an external evaluation.

1. **FP-Devicer technical whitepaper, version 2.0.3:** `whitepaper.md`; [previous whitepaper \(PDF\)](#).
2. **NEXT principles and roadmap:** `next-manifesto.md`; research aspirations are not statements of shipped capability.
3. **Release scope and October 4 verification checkpoint:** `docs/releases.md`.
4. **Observation and comparison implementation:** `src/next/index.ts`.
5. **Signal contracts:** `docs/next/domain-model-plan.md`, `src/next/signal-profiles.ts`, and `src/next/model-adapters.ts`.
6. **Calibration and adaptation:** `src/next/calibration.ts`, `src/next/adaptation.ts`, and `docs/next/uncertainty.md`.
7. **Retrieval, stores, artifacts, and governance:** `src/next/retrieval.ts`, `src/next/storage.ts`, `src/next/artifacts.ts`, and `src/next/governance.ts`.
8. **Evaluation machinery:** `src/next/evaluation.ts` and `src/next/benchmark.ts`.
9. **Compatibility contract and runtime boundaries:** `docs/compat/v2-contract.md` and `docs/next/architecture.md`.
10. **Source repository:** github.com/gatewaycorporate/fp-devicer.
11. **Generated API reference:** gatewaycorporate.github.io/fp-devicer.

Licensing and terms are provided in `license.txt` and `terms-of-service.md`.